

Sentiment Analysis of Flipkart Product Reviews Using NLP and Machine Learning

From:

Jianna Robin

24UBC135

To:

Ms. Roshni Rajendran

Department of
computer applications

Google Colab link:

[Click here to view Google Colab Notebook](#)

INTRODUCTION

Online Shopping has become a normal part of our everyday lives. Once in for a product many customers do check the ratings the product has received from others. With these ratings it is easy to know if the customers have purchased the product with satisfaction, disappointment or neutral feelings.

But when you do have thousands of product reviews reading all these individual reviews and categorizing them it is laborious and Time consuming. In such scenarios NLP which is Natural Language Processing can be an assistance. NLP refers to a field in Computer science, specifically in the area of Artificial intelligence, focused on enabling computers to interpret and use human language.

Here, in this project we use NLP and machine learning to analyze the Flipkart product reviews and extract the sentiment of these product reviews. This analysis can be categorized into three: Positive, Neutral and Negative sentiments.

At the initial level data needs to be cleaned up. After which TF-IDF can be applied to the reviews which gives the Numerical form of the reviews. Three different Machine Learning models namely Logistic Regression, Naive Bayes, linear SVM were used and evaluated with their corresponding performance scores and the best working model is utilized to predict the new customer reviews' sentiment.

PROBLEM STATEMENT

Flipkart contains a large number of customer reviews for different products. These reviews contain useful information about customer satisfaction and product quality.

The main problem is that manually checking thousands of reviews takes a lot of time. So, in this project, we try to create a system that can automatically understand the sentiment of a customer review.

The main problem addressed in this project is to classify Flipkart product reviews as Positive, Neutral, or Negative using NLP and machine learning techniques.

OBJECTIVES

The main objectives of this project are:

- To analyze customer reviews collected from flipkart
- To clean the review data and remove unnecessary information.
- To preprocess the review text so that it can be used for machine learning.
- To convert the review text into numerical features using TF-IDF.
- To train different machine learning models for sentiment classification.
- To compare the performance of the models.
- To identify the best-performing model.
- To test the final model using new customer reviews.
- To evaluate how well the model classifies different sentiment categories.

WHY THIS PROBLEM IS RELEVANT

Customer reviews help the consumer and the seller alike. To the consumer, customer reviews can aid them to determine whether or not a product is worth purchasing, and to the seller, they can offer insight to customer satisfaction.

Suppose many negative customer reviews about a product indicate that there is a problem with that product's quality, and then sellers are alerted to a issue that needs to be fixed. Because customer reviews are given as text, they can be efficiently analyzed using NLP tools, which allows automatic feedback extraction. Sentiment analysis of product reviews thus provides an example of a real-world application of NLP.

DATASET

Dataset Used

For this project, we used the **Flipkart Product Reviews with Sentiment Dataset**.

The dataset contains customer reviews collected from Flipkart along with their corresponding sentiment labels.

Dataset Attributes

The main columns in the dataset are:

Column	Description
product_name	Name of the product
product_price	Price of the product
Rate	Rating given by the customer
Review	Customer's detailed review
Summary	Short summary of the review
Sentiment	Sentiment of the review

The Review column is mainly used as the input text, while Sentiment is the target column.

Sentiment Classes

The reviews are divided into three categories:

- **Positive** – The customer is generally satisfied with the product.
- **Neutral** – The review expresses an average, mixed, or neutral opinion.
- **Negative** – The customer is dissatisfied with the product.

METHODOLOGY

The project follows a simple step-by-step process.

Flipkart Dataset



Data Cleaning



Text Preprocessing



TF-IDF Feature Extraction



Train-Test Split



Machine Learning Models



Prediction



Evaluation



Model Comparison



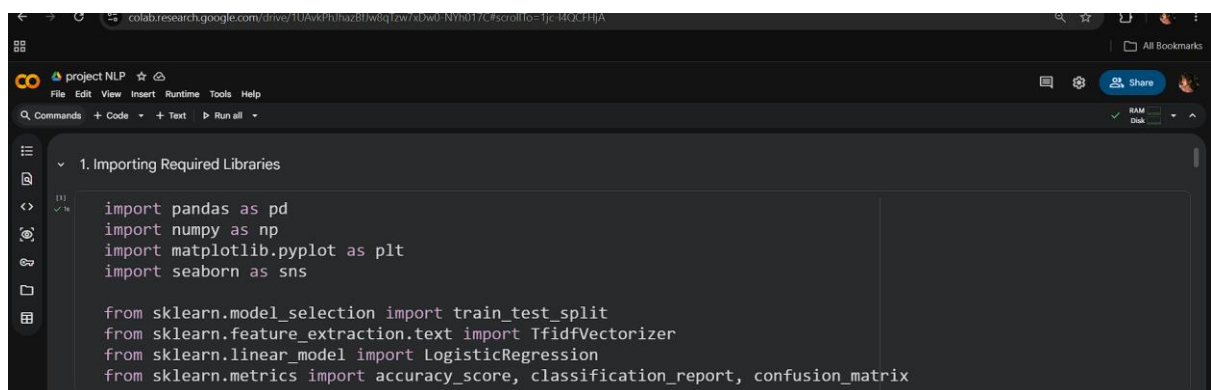
Final Model

The assignment expects the project to explain the relevant NLP technique, dataset preparation, model selection, implementation, testing, and result analysis.

IMPLEMENTATION

Importing Required Libraries

we need to import the Python libraries required for handling the dataset, preprocessing text, training the models, and displaying graphs.



```

1. Importing Required Libraries

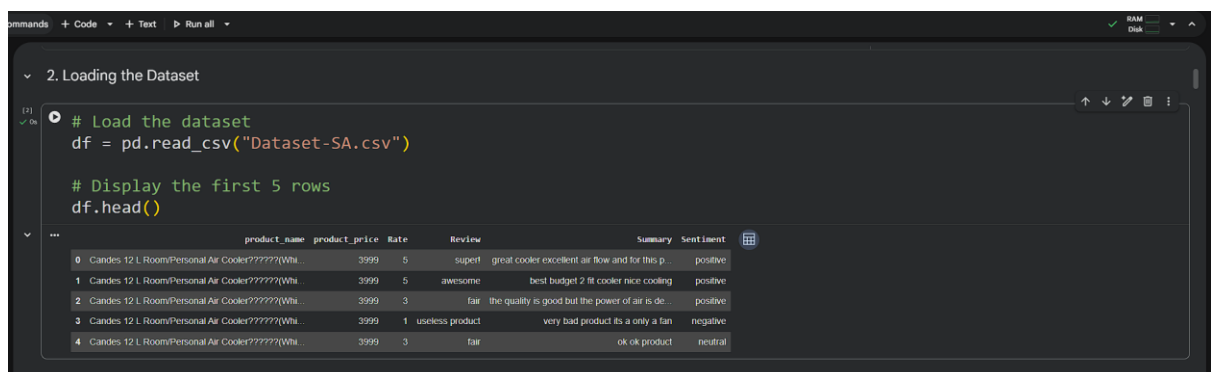
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
  
```

All the required libraries were imported successfully. These libraries are used for data handling, visualization, text feature extraction, model training, and evaluation.

Loading the Dataset

The dataset is stored in a CSV file named Dataset-SA.csv. We use Pandas to read this file and store it in a DataFrame called df.



```

2. Loading the Dataset

# Load the dataset
df = pd.read_csv("Dataset-SA.csv")

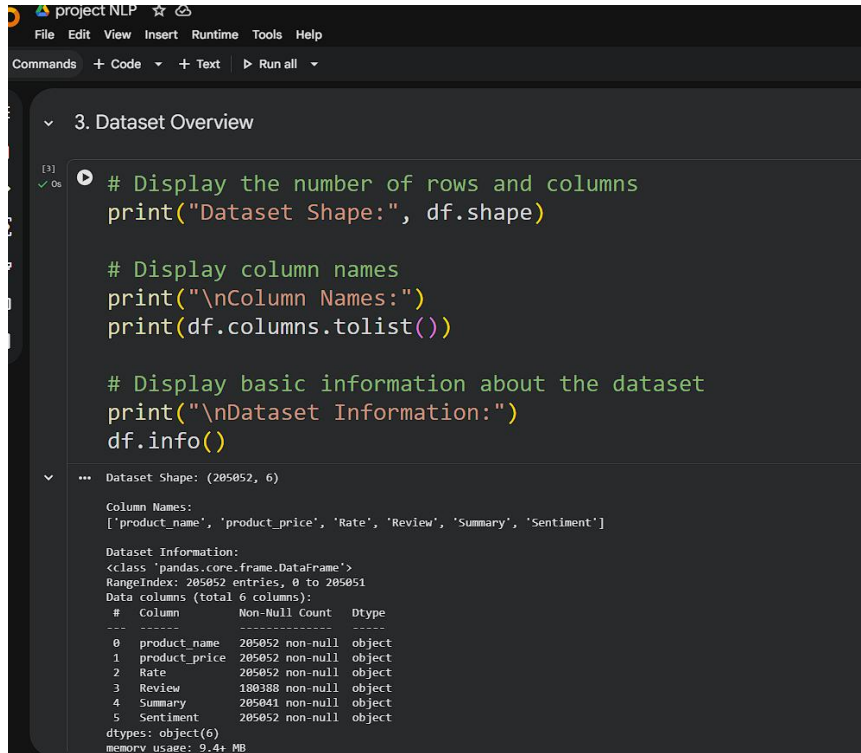
# Display the first 5 rows
df.head()
  
```

	product_name	product_price	Rate	Review	Summary	Sentiment
0	Candes 12 L Room/Personal Air Cooler?????(Whi...	3999	5	super great cooler excellent air flow and for this p...		positive
1	Candes 12 L Room/Personal Air Cooler?????(Whi...	3999	5	awesome best budget 2 lit cooler nice cooling		positive
2	Candes 12 L Room/Personal Air Cooler?????(Whi...	3999	3	fair the quality is good but the power of air is de...		positive
3	Candes 12 L Room/Personal Air Cooler?????(Whi...	3999	1	useless product very bad product its a only a fan		negative
4	Candes 12 L Room/Personal Air Cooler?????(Whi...	3999	3	fair ok ok product		neutral

The dataset was loaded successfully, and the first few records were checked to make sure the data was read correctly.

Understanding the Dataset

After loading the dataset, we need to understand what kind of data it contains. We check the dataset size, column names, and data types.



```

3. Dataset Overview

[3] ✓ Os
# Display the number of rows and columns
print("Dataset Shape:", df.shape)

# Display column names
print("\nColumn Names:")
print(df.columns.tolist())

# Display basic information about the dataset
print("\nDataset Information:")
df.info()

... Dataset Shape: (285052, 6)

Column Names:
['product_name', 'product_price', 'Rate', 'Review', 'Summary', 'Sentiment']

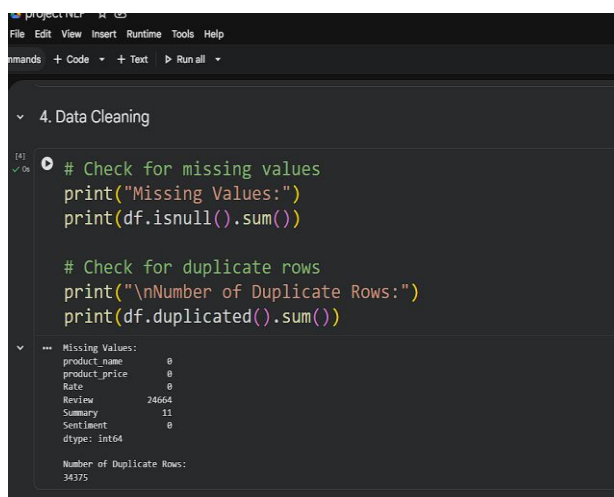
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 285052 entries, 0 to 285051
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ---
 0   product_name    285052 non-null object
 1   product_price   285052 non-null object
 2   Rate            285052 non-null object
 3   Review          180388 non-null object
 4   Summary         285041 non-null object
 5   Sentiment       285052 non-null object
dtypes: object(6)
memory usage: 9.4+ MB

```

This step helped us understand the overall structure of the dataset, including the number of records, columns, and the type of information stored in each column.

Checking Missing Values and Duplicates

Before using the data for machine learning, we need to check whether any values are missing and whether the same records appear more than once.



```

4. Data Cleaning

[4] ✓ Os
# Check for missing values
print("Missing Values:")
print(df.isnull().sum())

# Check for duplicate rows
print("\nNumber of Duplicate Rows:")
print(df.duplicated().sum())

... Missing Values:
product_name    0
product_price   0
Rate            0
Review          24664
Summary         11
Sentiment       0
dtype: int64

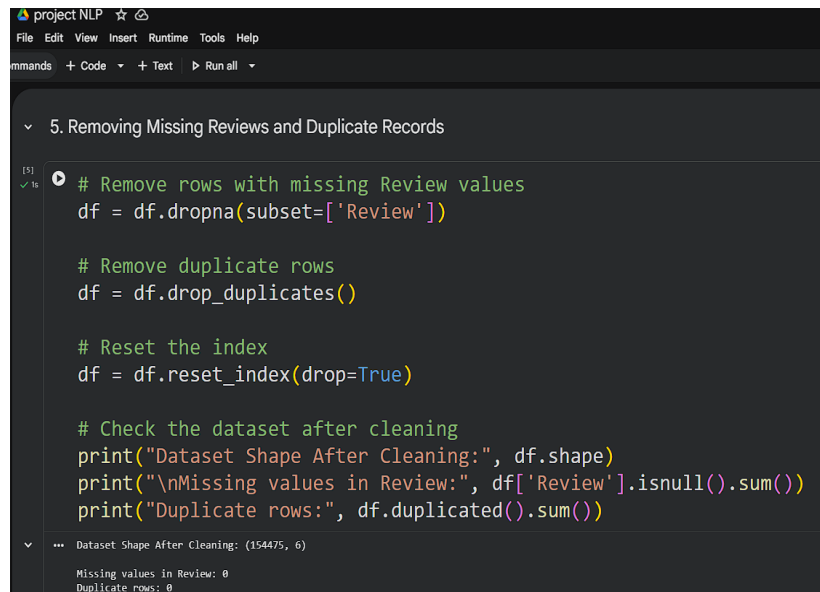
Number of Duplicate Rows:
34375

```

The dataset contained a considerable number of duplicate records and some missing values. Since the Review column is important for our project, records without a review cannot be directly used for sentiment analysis.

Data Cleaning

To make the dataset more suitable for our model, we remove rows where the Review is missing. We also remove duplicate records.



The screenshot shows a Jupyter Notebook interface with a dark theme. The title bar says 'project NLP'. The menu bar includes 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu bar is a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all'. The notebook content is titled '5. Removing Missing Reviews and Duplicate Records'. It contains the following Python code:

```
[5]: # Remove rows with missing Review values
df = df.dropna(subset=['Review'])

# Remove duplicate rows
df = df.drop_duplicates()

# Reset the index
df = df.reset_index(drop=True)

# Check the dataset after cleaning
print("Dataset Shape After Cleaning:", df.shape)
print("\nMissing values in Review:", df['Review'].isnull().sum())
print("Duplicate rows:", df.duplicated().sum())
```

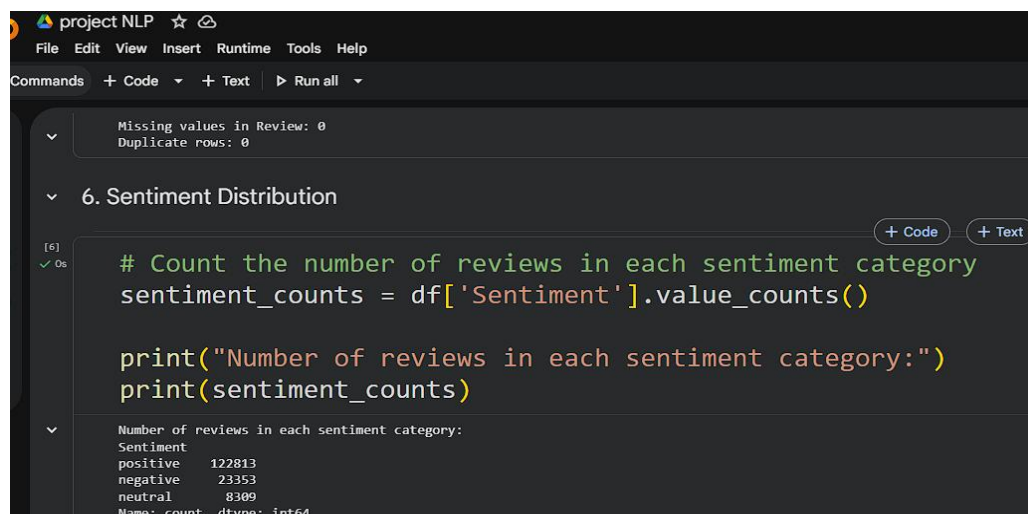
Below the code, the output is displayed:

```
Dataset Shape After Cleaning: (154475, 6)
Missing values in Review: 0
Duplicate rows: 0
```

The missing review records and duplicate rows were removed successfully. This gave us a cleaner dataset for the next stages of the project.

Sentiment Distribution

we check how many reviews belong to each sentiment category. This helps us understand the distribution of Positive, Neutral, and Negative reviews.



The screenshot shows a Jupyter Notebook interface with a dark theme. The title bar says 'project NLP'. The menu bar includes 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu bar is a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all'. The notebook content is titled '6. Sentiment Distribution'. It contains the following Python code:

```
[6]: # Count the number of reviews in each sentiment category
sentiment_counts = df['Sentiment'].value_counts()

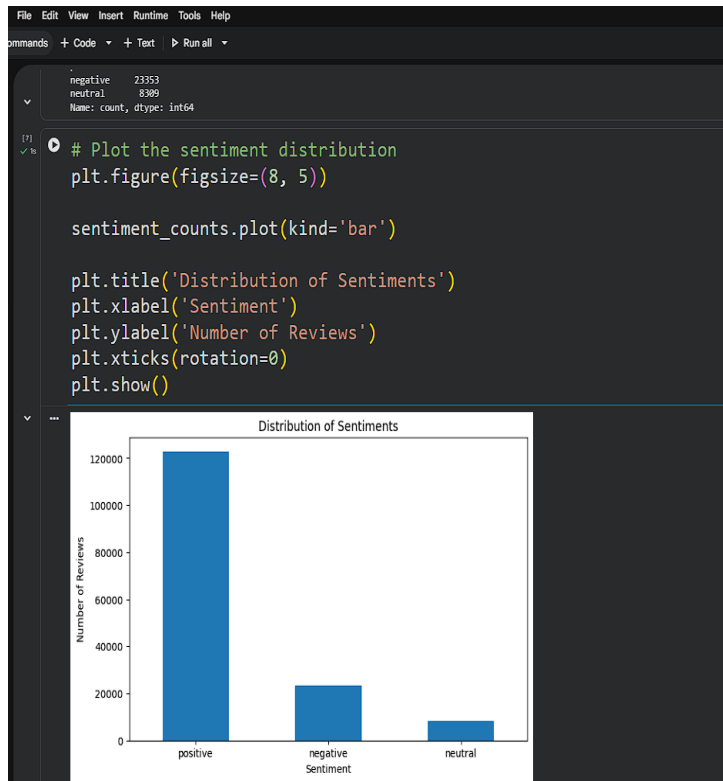
print("Number of reviews in each sentiment category:")
print(sentiment_counts)
```

Below the code, the output is displayed:

```
Number of reviews in each sentiment category:
Sentiment
positive    122813
negative    23353
neutral      8309
Name: count, dtype: int64
```


The number of reviews in each sentiment category was calculated to understand how the three classes are distributed in the dataset.

Visualizing Sentiment Distribution



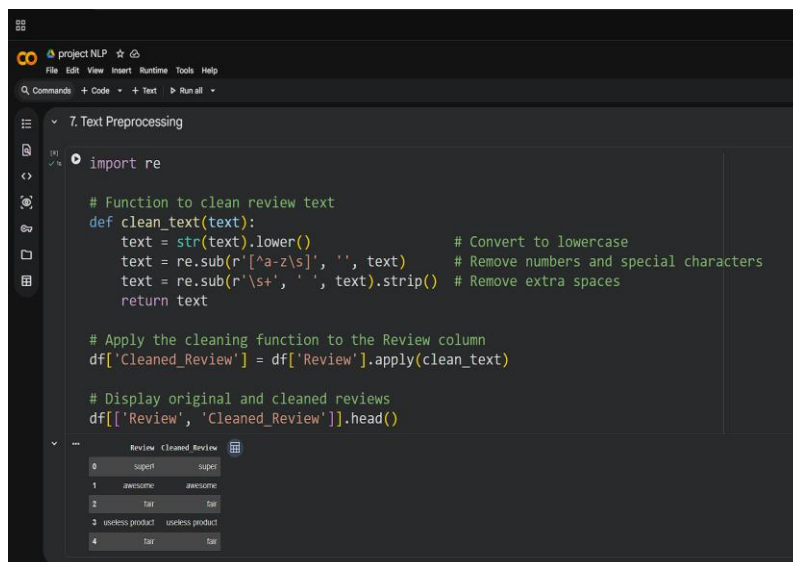
The bar graph gives a clear visual representation of the number of Positive, Neutral, and Negative reviews in the cleaned dataset.

Text Preprocessing

Customer reviews are written in natural language, so they may contain capital letters, punctuation marks, numbers, and unnecessary spaces. Before giving the reviews to the machine learning model, we clean the text.

In this project, we:

- Convert the text to lowercase.
- Remove unnecessary characters.
- Remove extra spaces.



```

import re

# Function to clean review text
def clean_text(text):
    text = str(text).lower()          # Convert to lowercase
    text = re.sub(r'[^\w\s]', '', text) # Remove numbers and special characters
    text = re.sub(r'\s+', ' ', text).strip() # Remove extra spaces
    return text

# Apply the cleaning function to the Review column
df['Cleaned_Review'] = df['Review'].apply(clean_text)

# Display original and cleaned reviews
df[['Review', 'Cleaned_Review']].head()

```

	Review	Cleaned_Review
0	super!	super
1	awesome!	awesome
2	far	far
3	useless product	useless product
4	far	far

The review text was successfully cleaned. Converting the text into a consistent format makes it easier for the machine learning model to process the reviews.

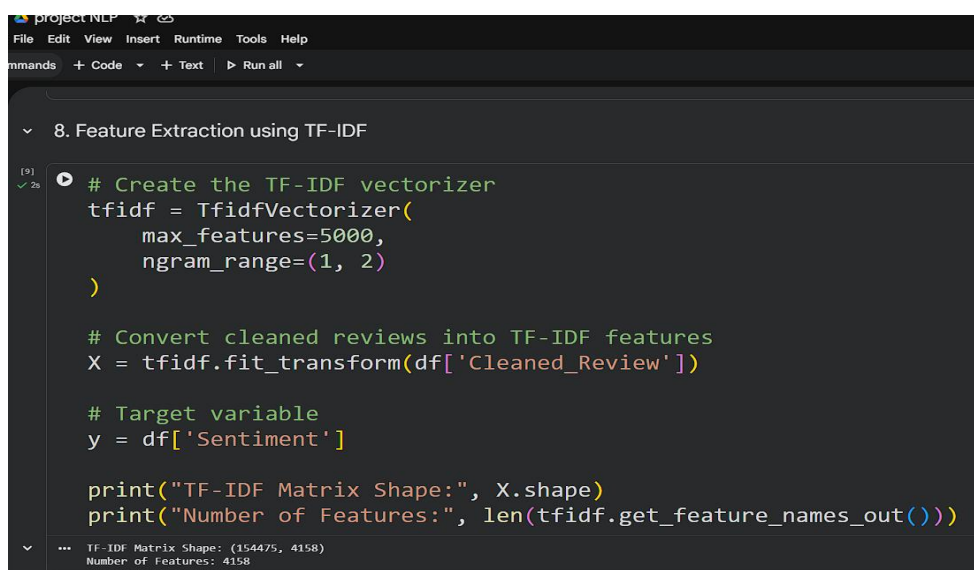
Feature Extraction Using TF-IDF

Machine learning models cannot directly understand sentences or words. Therefore, we need to convert the cleaned review text into numbers.

For this purpose, we use **TF-IDF (Term Frequency-Inverse Document Frequency)**.

TF-IDF gives higher importance to words that are useful for distinguishing between different reviews.

In our implementation, we use a maximum of **5000 features** and consider both single words and two-word combinations.



```

# 8. Feature Extraction using TF-IDF

# Create the TF-IDF vectorizer
tfidf = TfidfVectorizer(
    max_features=5000,
    ngram_range=(1, 2)
)

# Convert cleaned reviews into TF-IDF features
X = tfidf.fit_transform(df['Cleaned_Review'])

# Target variable
y = df['Sentiment']

print("TF-IDF Matrix Shape:", X.shape)
print("Number of Features:", len(tfidf.get_feature_names_out()))

```

```

TF-IDF Matrix Shape: (154475, 4158)
Number of Features: 4158

```

The cleaned reviews were successfully converted into numerical TF-IDF features. These features can now be used as input for the machine learning models.

Splitting the Dataset

We divide the dataset into training and testing data.

- 80% of the data is used for training.
- 20% is used for testing.

The model learns patterns from the training data and is then tested using data that it has not seen before.

9. Train-Test Split

```
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

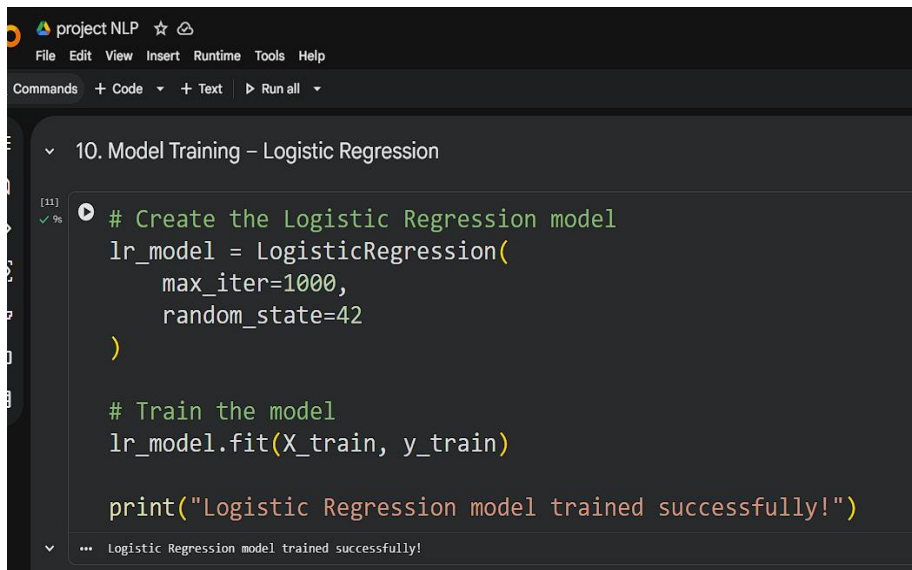
print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

... Training samples: 123580
Testing samples: 30895

The dataset was divided into training and testing sets using an 80:20 ratio. Stratified splitting was used to maintain a similar sentiment distribution in both sets.

Training Logistic Regression

The first model used in the project is Logistic Regression. It is a simple and commonly used classification algorithm and works well with text features such as TF-IDF.



```
project NLP
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

10. Model Training - Logistic Regression

[11] ✓ % # Create the Logistic Regression model
lr_model = LogisticRegression(
    max_iter=1000,
    random_state=42
)

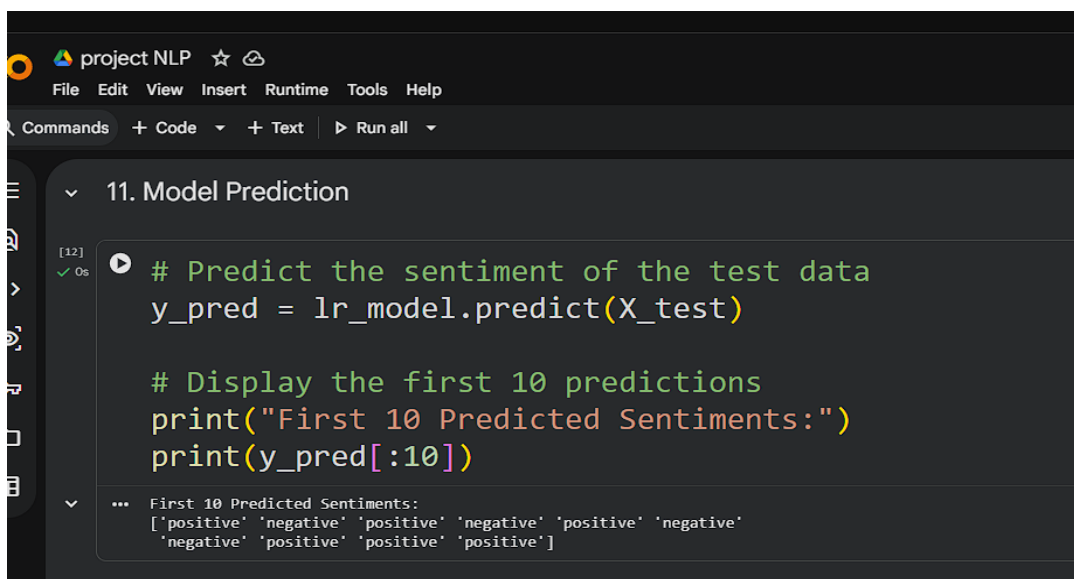
# Train the model
lr_model.fit(X_train, y_train)

print("Logistic Regression model trained successfully!")

... Logistic Regression model trained successfully!
```

The Logistic Regression model was successfully trained using the TF-IDF features extracted from the customer reviews.

Logistic Regression Prediction



```
project NLP
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

11. Model Prediction

[12] ✓ Os # Predict the sentiment of the test data
y_pred = lr_model.predict(X_test)

# Display the first 10 predictions
print("First 10 Predicted Sentiments:")
print(y_pred[:10])

... First 10 Predicted Sentiments:
['positive' 'negative' 'positive' 'negative' 'positive' 'negative'
 'negative' 'positive' 'positive' 'positive']
```

The trained Logistic Regression model generated sentiment predictions for the test reviews.

Logistic Regression Evaluation

```

12. Model Evaluation

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", round(accuracy * 100, 2), "%")

# Display classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

Accuracy: 90.26 %

Classification Report:
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use 'zero_division' parameter to control the handling of labels with no predicted samples.
warn_prf(average, modifier, f"[metric.capitalize()] is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use 'zero_division' parameter to control the handling of labels with no predicted samples.
warn_prf(average, modifier, f"[metric.capitalize()] is", len(result))
precision    recall  f1-score   support

negative    0.84    0.78    0.81    4678
neutral     0.80    0.80    0.80    1662
positive    0.91    0.99    0.95   24563

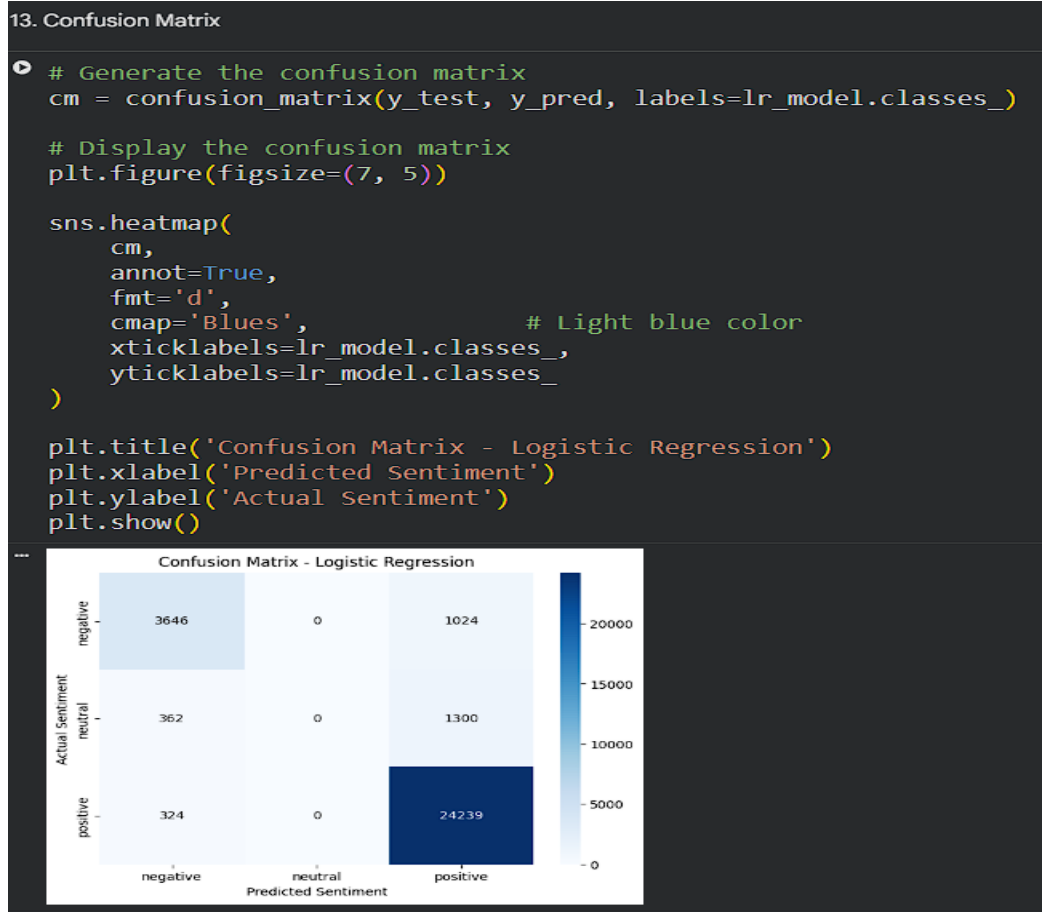
accuracy    0.90    0.90    0.90   30895
macro avg   0.58    0.59    0.59   30895
weighted avg 0.85    0.90    0.88   30895

/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use 'zero_division' parameter to control the handling of labels with no predicted samples.
warn_prf(average, modifier, f"[metric.capitalize()] is", len(result))

```

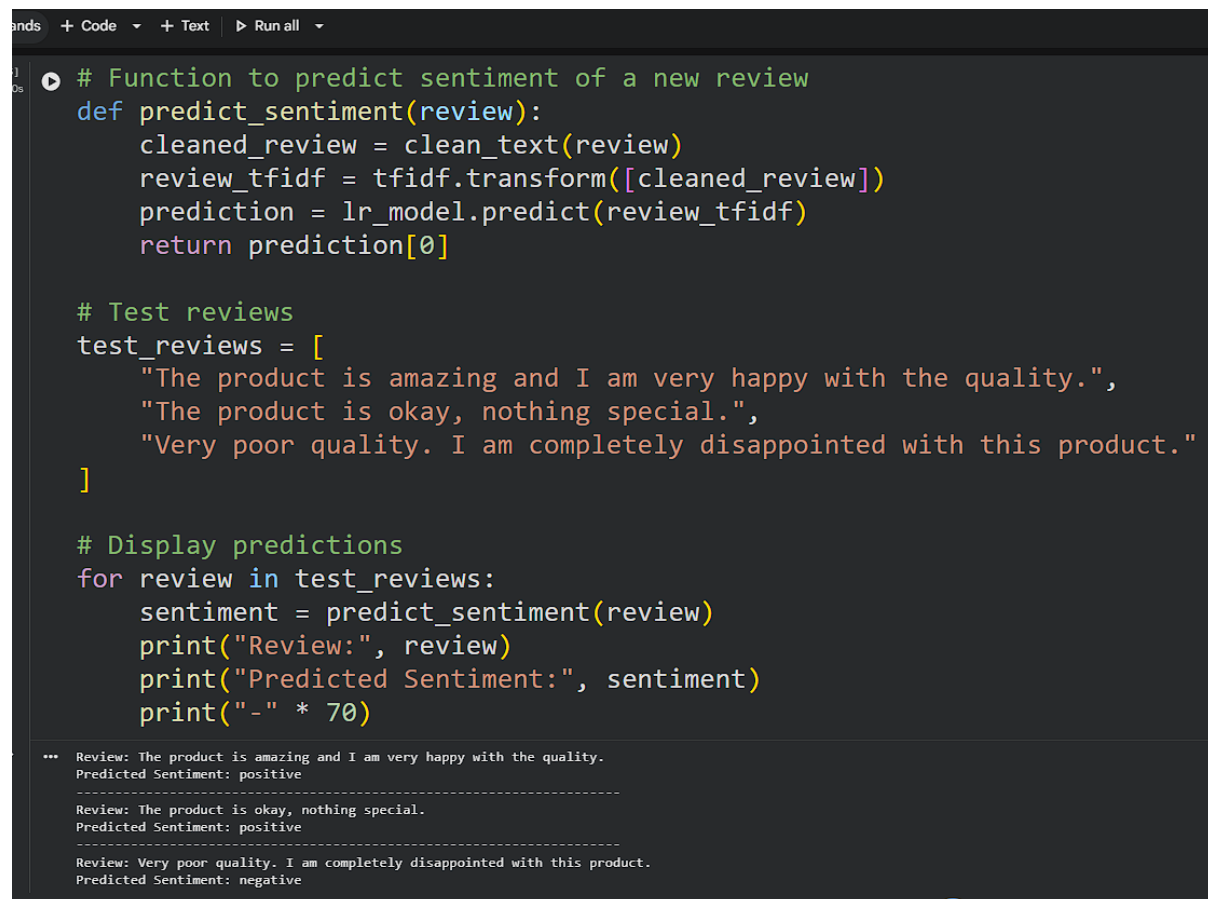
Logistic Regression achieved an accuracy of 90.26%. The classification report gives a more detailed view of how the model performed for each sentiment class.

Logistic Regression Confusion Matrix



The confusion matrix helps us see which reviews were classified correctly and which were classified incorrectly. It also shows which sentiment categories are more difficult for the model to distinguish.

Testing with New Reviews



```
ands + Code + Text ▶ Run all
# Function to predict sentiment of a new review
def predict_sentiment(review):
    cleaned_review = clean_text(review)
    review_tfidf = tfidf.transform([cleaned_review])
    prediction = lr_model.predict(review_tfidf)
    return prediction[0]

# Test reviews
test_reviews = [
    "The product is amazing and I am very happy with the quality.",
    "The product is okay, nothing special.",
    "Very poor quality. I am completely disappointed with this product."
]

# Display predictions
for review in test_reviews:
    sentiment = predict_sentiment(review)
    print("Review:", review)
    print("Predicted Sentiment:", sentiment)
    print("-" * 70)

... Review: The product is amazing and I am very happy with the quality.
    Predicted Sentiment: positive
    -----
    Review: The product is okay, nothing special.
    Predicted Sentiment: positive
    -----
    Review: Very poor quality. I am completely disappointed with this product.
    Predicted Sentiment: negative
```

The model was tested with new reviews to check whether it could classify customer opinions that were not part of the original test dataset.

Comparison of Machine Learning Models

15. Comparing Machine Learning Models

```

from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC

# Train Naive Bayes
nb_model = MultinomialNB()
nb_model.fit(X_train, y_train)

# Train Linear SVM
svm_model = LinearSVC(random_state=42)
svm_model.fit(X_train, y_train)

# Predictions
nb_pred = nb_model.predict(X_test)
svm_pred = svm_model.predict(X_test)

# Compare accuracies
print("Model Performance")
print("-" * 40)

print("Logistic Regression Accuracy:",
      round(accuracy_score(y_test, y_pred) * 100, 2), "%")

print("Naive Bayes Accuracy:",
      round(accuracy_score(y_test, nb_pred) * 100, 2), "%")

print("Linear SVM Accuracy:",
      round(accuracy_score(y_test, svm_pred) * 100, 2), "%")

```

```

--- Model Performance
-----
Logistic Regression Accuracy: 90.26 %
Naive Bayes Accuracy: 89.44 %
Linear SVM Accuracy: 90.27 %

```

All three models produced good accuracy. However, Linear SVM achieved the highest accuracy of 90.27%, which was slightly higher than Logistic Regression and Naive Bayes.

Final Model – Linear SVM

16. Final Model Evaluation – Linear SVM

```

# Generate the classification report for Linear SVM
print("Linear SVM Classification Report:")
print(classification_report(y_test, svm_pred))

```

```

Linear SVM Classification Report:
      precision    recall  f1-score   support

negative    0.84      0.78      0.81      4670
neutral     0.25      0.00      0.00      1662
positive    0.91      0.99      0.95     24563

accuracy          0.67          0.59          0.90     30895
macro avg         0.67          0.59          0.88     30895
weighted avg         0.87          0.90          0.88     30895

```

The classification report was used to check the precision, recall, and F1-score of the final Linear SVM model for Positive, Neutral, and Negative reviews.

Linear SVM Confusion Matrix

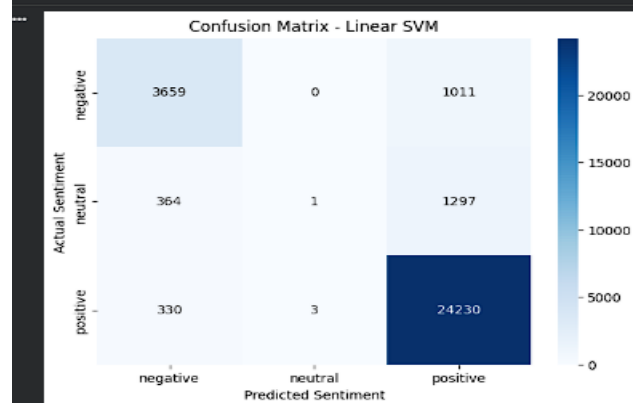
7. Confusion Matrix – Linear SVM

```
# Generate the confusion matrix for Linear SVM
cm_svm = confusion_matrix(y_test, svm_pred, labels=svm_model.classes_)

# Display the confusion matrix
plt.figure(figsize=(7, 5))

sns.heatmap(
    cm_svm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=svm_model.classes_,
    yticklabels=svm_model.classes_
)

plt.title('Confusion Matrix - Linear SVM')
plt.xlabel('Predicted Sentiment')
plt.ylabel('Actual Sentiment')
plt.show()
```



The confusion matrix shows how accurately the final model classified the three sentiment categories and where classification errors occurred.

Testing the Final Model

```

Testing New Customer Reviews - Linear SVM

# Function to predict sentiment using the final Linear SVM model
def predict_sentiment_svm(review):
    cleaned_review = clean_text(review)
    review_tfidf = tfidf.transform([cleaned_review])
    prediction = svm_model.predict(review_tfidf)
    return prediction[0]

# Test new customer reviews
test_reviews = [
    "The product quality is excellent and I am very happy with it.",
    "The product is okay, but nothing special.",
    "Very poor quality and completely disappointing."
]

# Display predictions
for review in test_reviews:
    sentiment = predict_sentiment_svm(review)

    print("Review:", review)
    print("Predicted Sentiment:", sentiment)
    print("-" * 70)

Review: The product quality is excellent and I am very happy with it.
Predicted Sentiment: positive
-----
Review: The product is okay, but nothing special.
Predicted Sentiment: positive
-----
Review: Very poor quality and completely disappointing.
Predicted Sentiment: negative
-----

```

The final Linear SVM model was able to take new customer reviews as input and predict their sentiment category.

Results and Discussion

The project produced good results for the sentiment classification task.

The accuracy of the three models is shown below:

Model	Accuracy
Logistic Regression	90.26%
Naive Bayes	89.44%
Linear SVM	90.27%

From the comparison, Linear SVM performed slightly better than the other two models.

Therefore, Linear SVM was selected as the final model.

LIMITATIONS

There are some limitations in the current project.

- The model mainly works with English review text.
- Sarcasm and indirect meanings can be difficult for the model to understand.
- TF-IDF mainly focuses on word occurrence and does not fully understand the context of a sentence.
- The model is trained using Flipkart reviews, so its performance may be different on reviews from other websites.
- Emojis and some special expressions are removed during preprocessing, even though they can sometimes provide useful sentiment information.
- Overall accuracy alone does not show the complete performance of the model, so precision, recall, and F1-score also need to be considered.

FUTURE SCOPE

The project can be improved in the future by using more advanced NLP techniques.

- Deep learning models such as **LSTM and GRU** can be used.
- Transformer-based models such as **BERT** can be explored.
- The system can be extended to support Malayalam and other Indian languages.
- Emojis and other sentiment-related symbols can be included during preprocessing.
- Aspect-based sentiment analysis can be added to identify opinions about specific features such as **quality, price, delivery, and packaging**.
- The project can be converted into a web application where users can enter a review and immediately get its predicted sentiment.

CONCLUSION

In this project, we developed a sentiment analysis system for Flipkart product reviews using NLP and machine learning.

The review data was first cleaned and preprocessed. TF-IDF was then used to convert the text into numerical features. Three machine learning algorithms were trained and compared.

The results were:

- Logistic Regression – **90.26%**
- Naive Bayes – **89.44%**
- Linear SVM – **90.27%**

Among these models, **Linear SVM gave the highest accuracy of 90.27%**. Therefore, it was selected as the final model.

The final system can classify new customer reviews into **Positive, Neutral, or Negative** categories.

Overall, this project helped demonstrate how NLP and machine learning can be used in a real-world situation to understand customer opinions from online product reviews.